



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

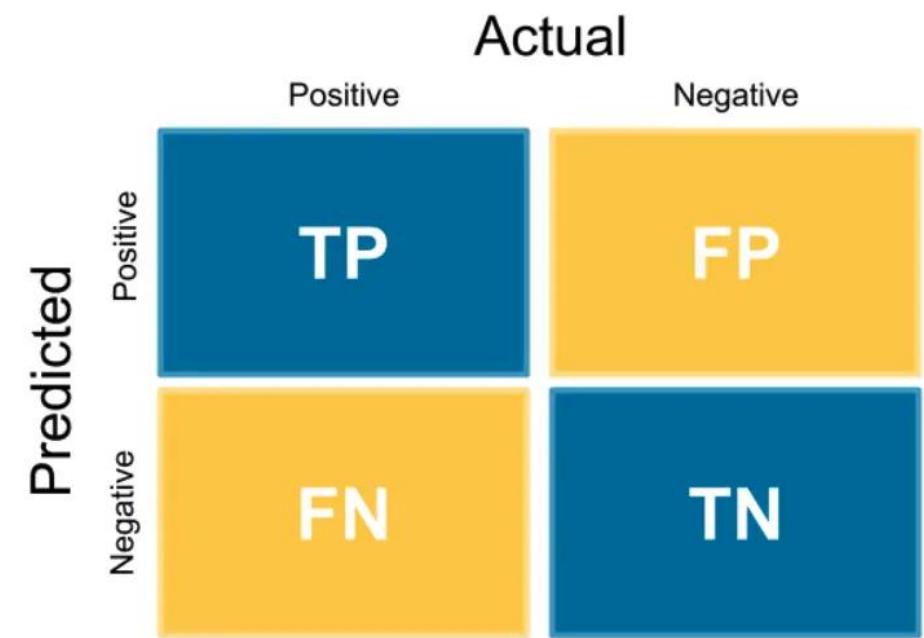
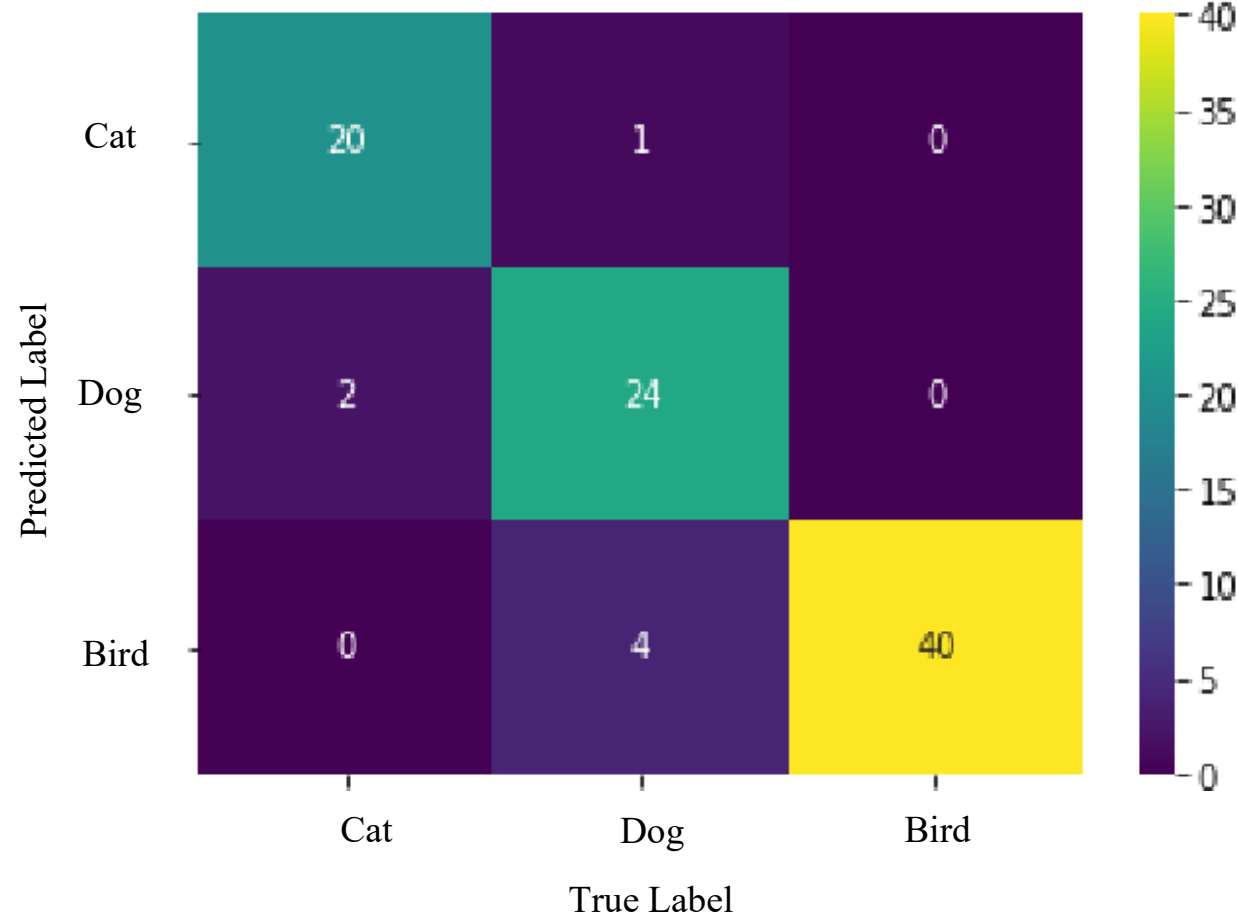
Ching-I Huang



Announcement

- 10/17 (Fri): HW1 due
- 10/21 (Tue), 15:30-18:00 Midterm Exam
- 10/23 (Thurs): No class

Confusion Matrix



True Positive (TP): Predicted as positive and actually true.
False Positive (FP): Predicted as positive but actually false.
False Negative (FN): Predicted as negative but actually true.
True Negative (TN): Predicted as negative and actually false.

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Number of Predictions (TP + TN + FP + FN)}}$$

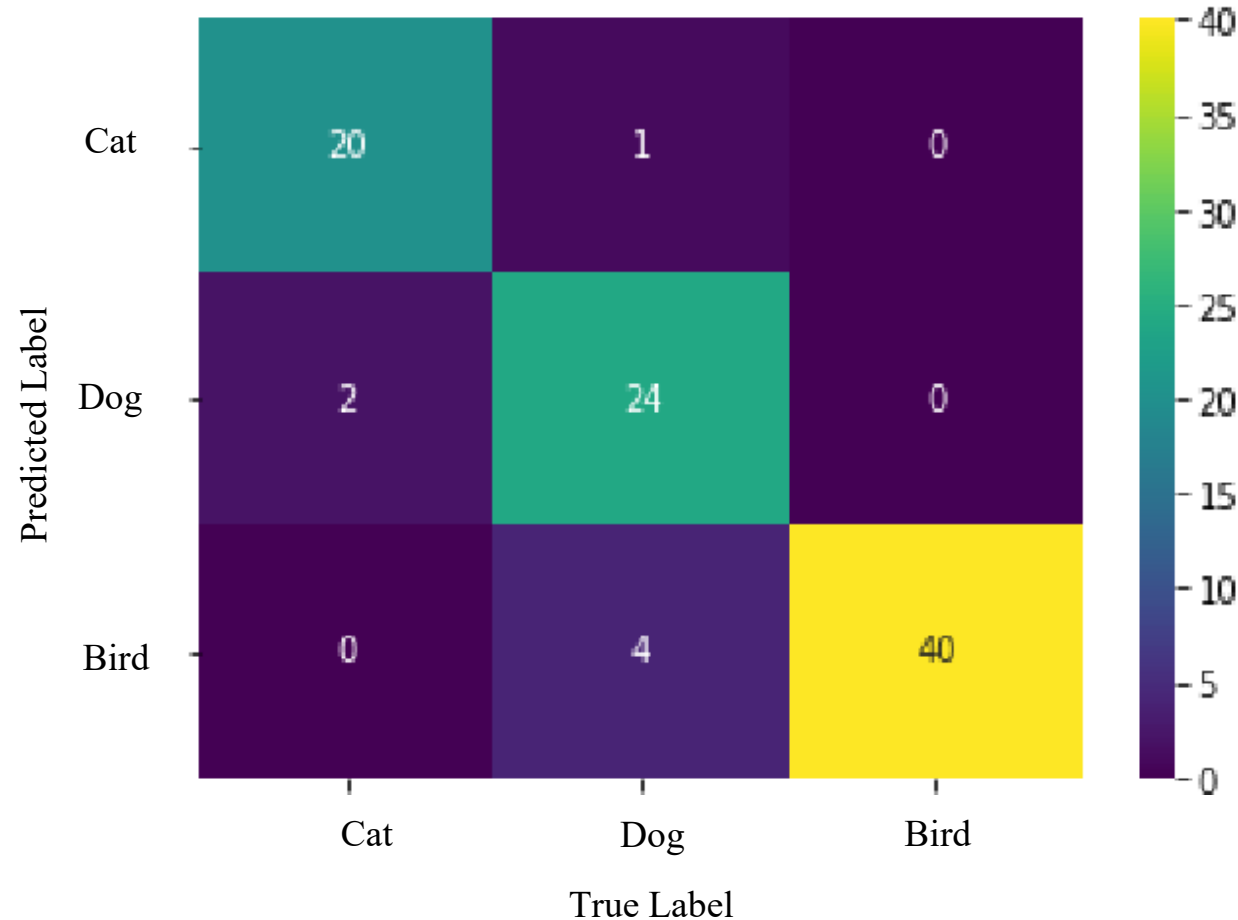
$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$



Confusion Matrix



	TP	FP	FN	TN
Cat	20	1+0	2+0	91-(20+2+1)
Dog	24	2+0	1+4	91-(24+2+5)
Bird	40	0+4	0+0	91-(4+0+40)

True Positive (TP): Predicted as positive and actually true.

False Positive (FP): Predicted as positive but actually false.

False Negative (FN): Predicted as negative but actually true.

True Negative (TN): Predicted as negative and actually false.

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Number of Predictions (TP + TN + FP + FN)}}$$

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$



Generalization

- Deep CNNs can learn complex hierarchical features, but with millions of parameters, they easily memorize training data.
→ overfitting
- Regularization (**improves generalization**)
 1. Control model capacity, e.g., early stopping or weight decay (penalizes large weights)
 2. Add randomness, e.g., bagging, dropout
- Bagging
 1. Creates multiple datasets by random resampling with replacement from the original training data
 2. Trains independent models on each dataset.
 3. Combines them (average/vote), reducing variance.



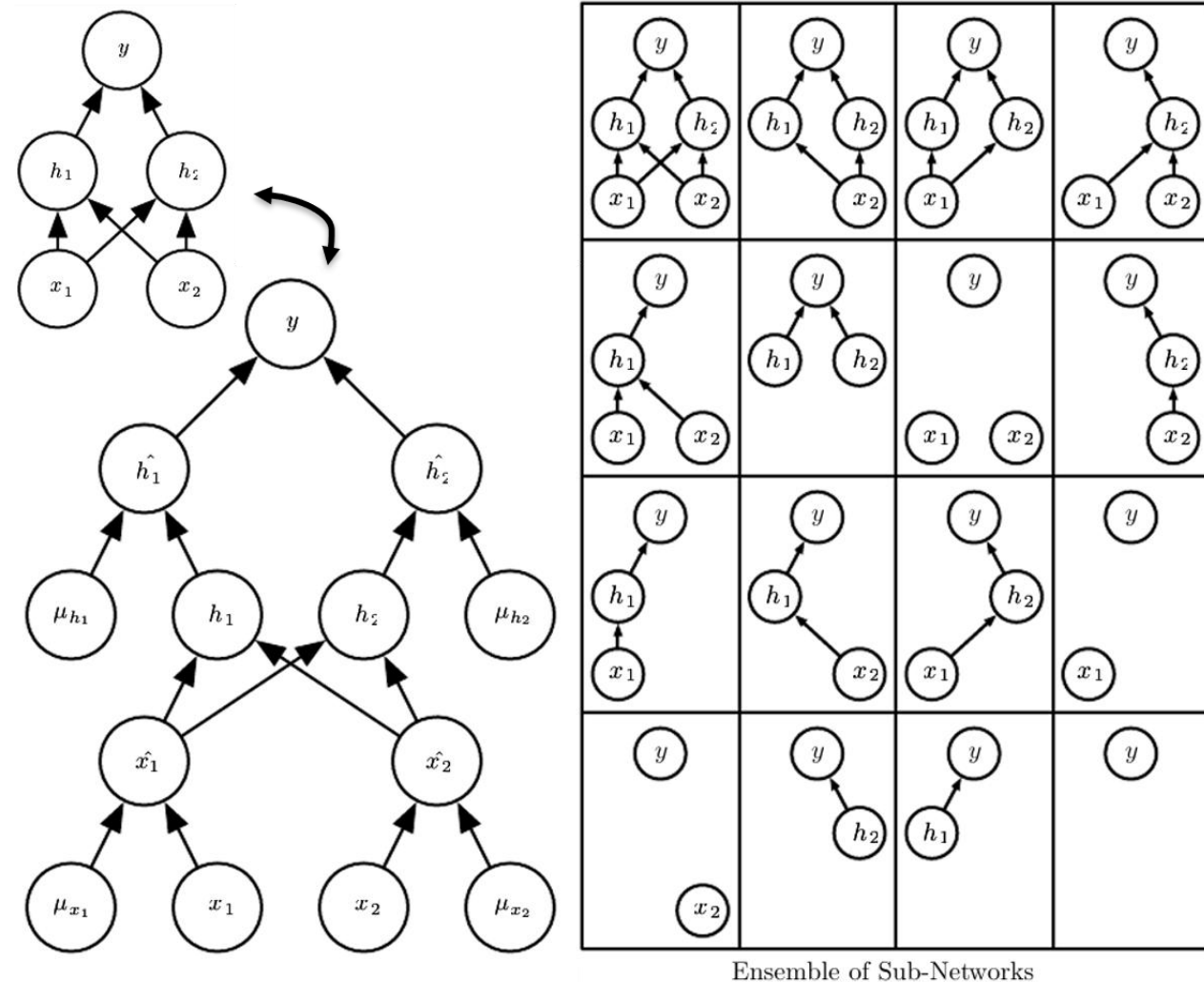
Dropout

- Bagging *over subnetworks*, with shared parameters and no need to store multiple models.
- Dropout achieves the same ensemble effect **within one network** by randomly deactivating neurons.

$$\arg \min_{\Theta} \mathbb{E}_{r \sim \text{Bernoulli}(\mu)} [C(\Theta; r)]$$

keep probability binary mask vector
loss computed when applying mask

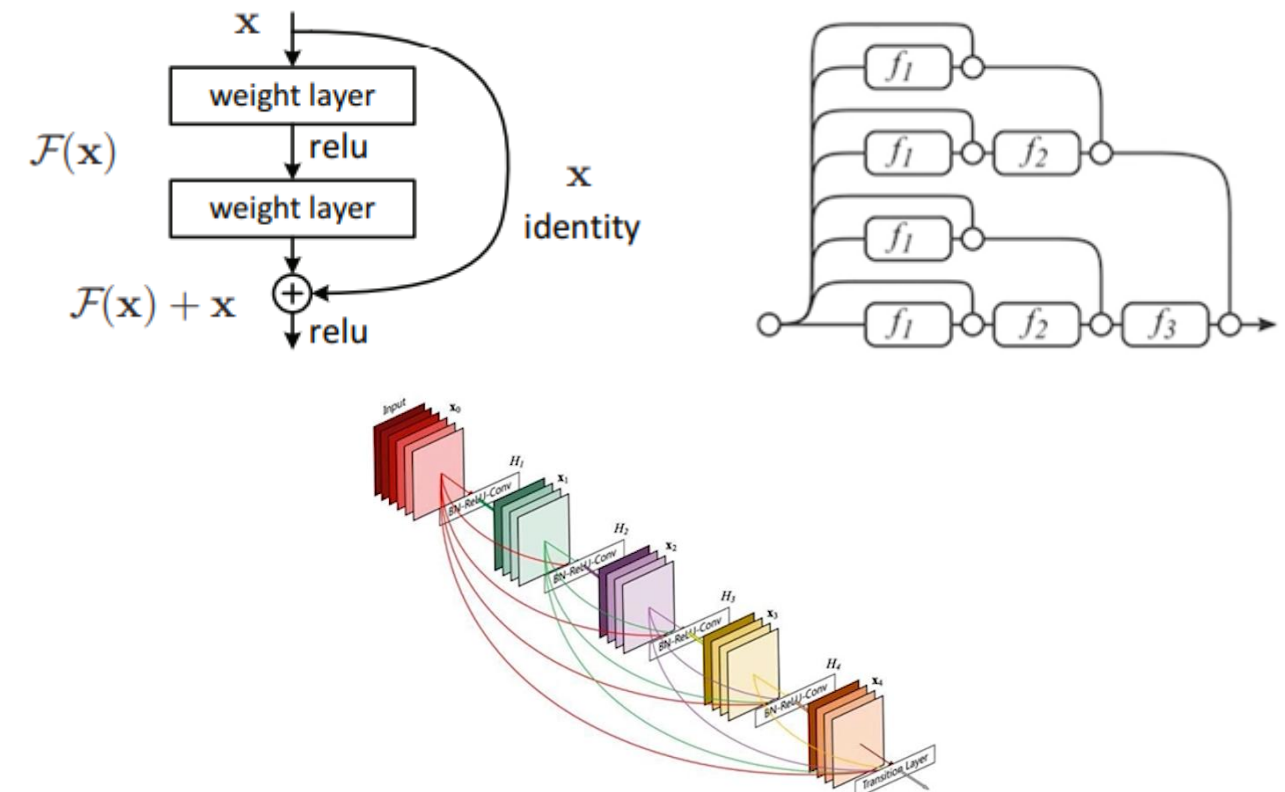
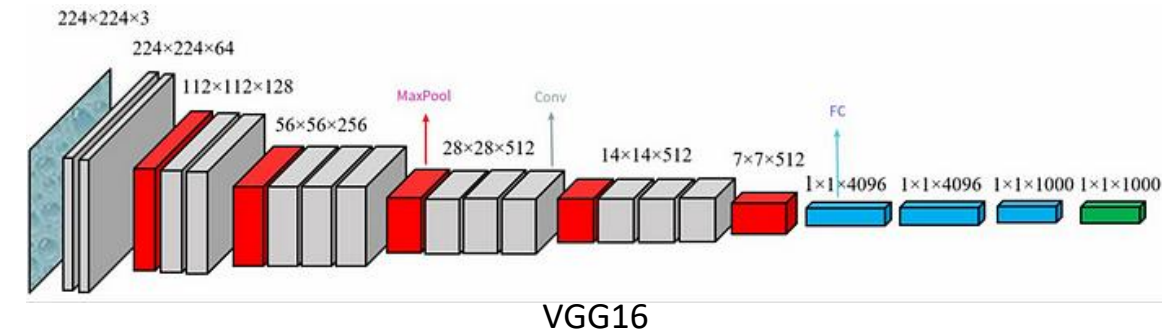
- Main points
 1. Variance reduction
 2. Co-adaptation prevention
 3. Robustness & redundancy
 4. Memory-efficient ensemble





Case Study

- VGG (2014): Replaced large filters (e.g., 5x5) with multiple smaller filters (3x3).
 1. Stacks of smaller filters provide the same effective receptive field as larger filters while reducing the number of parameters.
 2. Increases network depth, allowing it to learn more complex features without a significant computational cost increase.
- ResNet (2016): Residual learning with shortcut connections, allowing gradients to flow through deeper networks.
 1. Deep (152 layers)
 2. Batch normalization
 3. Fully convolutional network (without fully-connected layer at the end)
 4. Deeper CNN always perform equal or better to a shallower one (skip the layer if they are not helpful).
- DenseNet (2017): Dense connections where each layer receives inputs from all preceding layers within a block.
 1. Feature reuse and sharing across layers
 2. Mitigates vanishing gradients





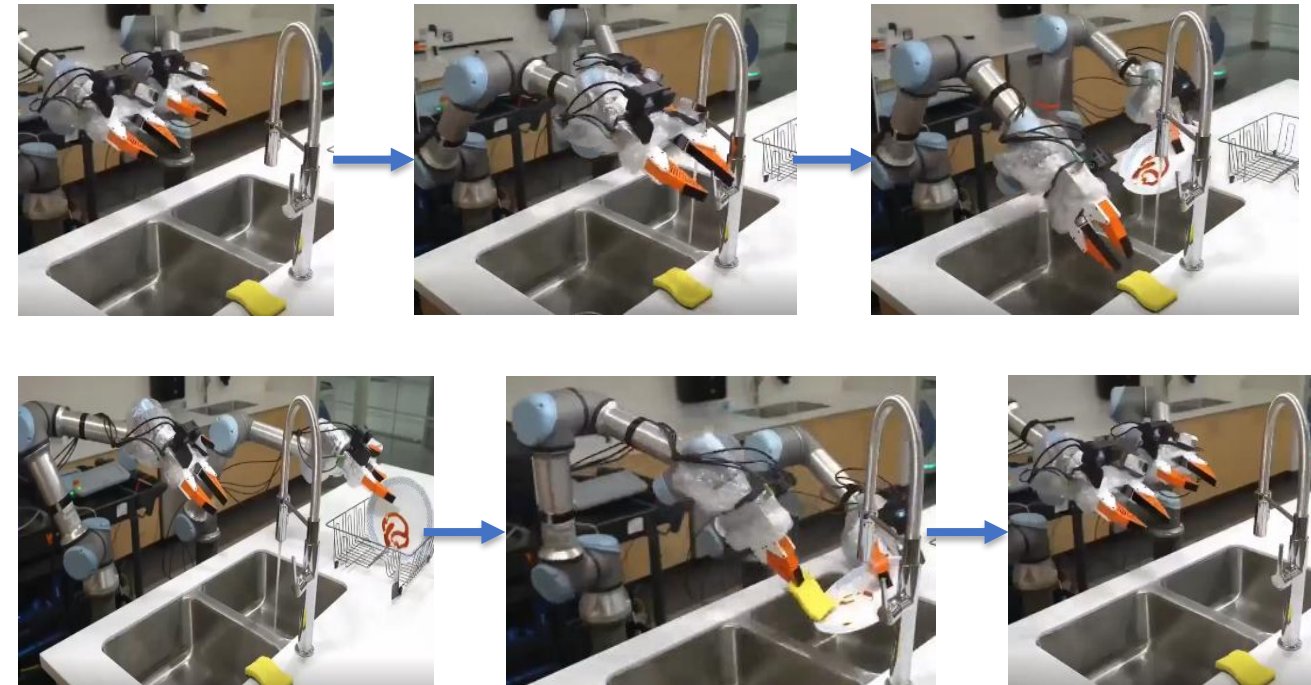
Sequential Data

- Data points come in order and successive point may be dependent.

1. Words in a sentence
2. Phonemes in a spoken word utterance
3. Page click
4. Frames of video
5. Autonomy driving

- Dataset: $X = \{X^{(n)}\}_n$

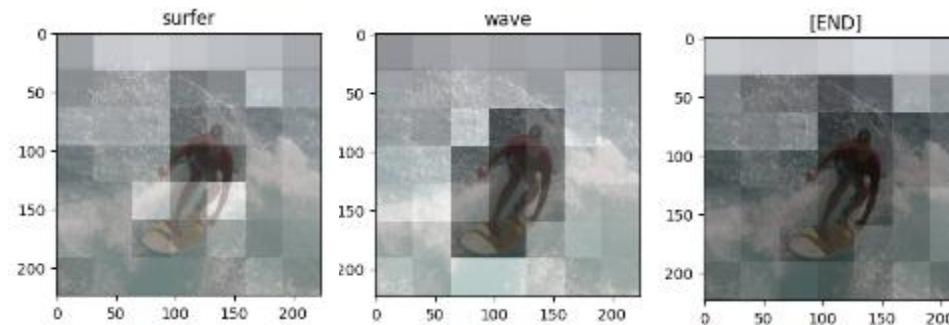
- Sequence $X^{(n)} = \left\{ \left(x^{(n,t)}, y^{(n,t)} \right) \right\}_t$





Other types

- It is not necessary for $x^{(t)}$ and $y^{(t)}$ to have a one-to-one correspondence.
 - One to Many: [Image Captioning](#): A surfer is surfing on a wave. [End]

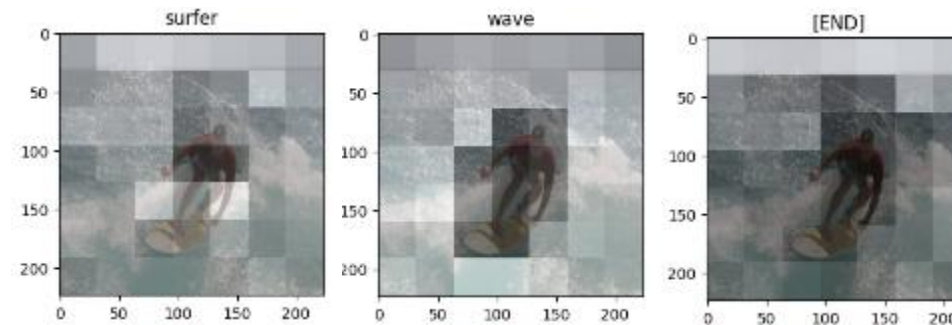


- Many to one: “**Congratulations!** NYCU **won** the **second place in Meichu Games**” → **Positive/Negative** ?
- Many to Many (Synced): Event-SLAM find out the key frame in the video.
- Many to Many (Unsynced): Machine Translation, Chatbot
- Recursive RNNs: NLP (nature language process) based on grammar rules
Congratulations! NYCU **won** the **second place in Meichu Games**
NYCU won the second place in Meichu Games. Congratulations!



Other types

- It is not necessary for $x^{(t)}$ and $y^{(t)}$ to have a one-to-one correspondence.
 - One to Many: [Image Captioning](#): A surfer is surfing on a wave. [End]

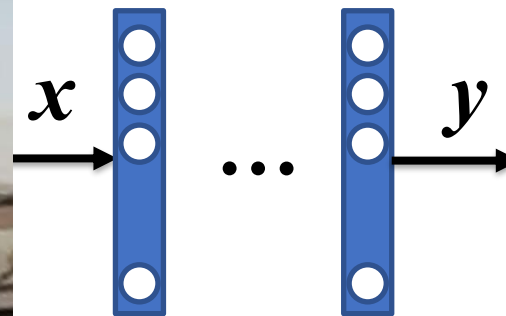


- Many to one: “**Congratulations!** NYCU **won** the **second place in Meichu Games**” → **Positive/Negative** ?
- Many to Many (Synced): Event-SLAM find out the key frame in the video.
- Many to Many (Unsynced): Machine Translation, Chatbot
- Recursive RNNs: NLP (nature language process) based on grammar rules
Congratulations! NYCU **won** the **second place in Meichu Games**
NYCU **won** the **second place in Meichu Games**. **Congratulations!**



Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)
 - $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

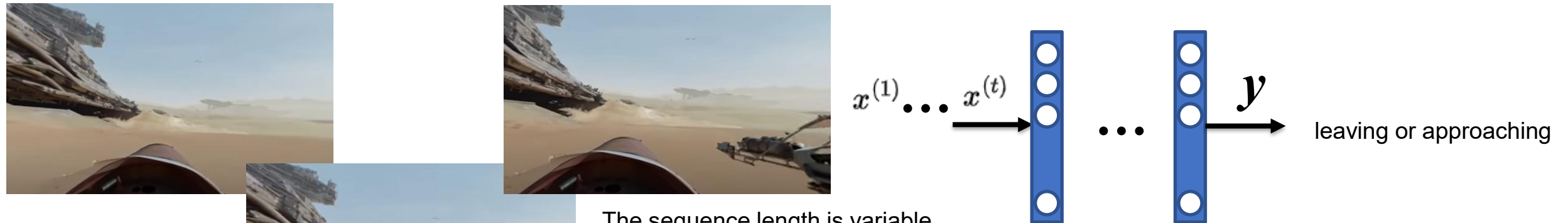


leaving or approaching



Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)
 - $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$



The sequence length is variable.



Memory?

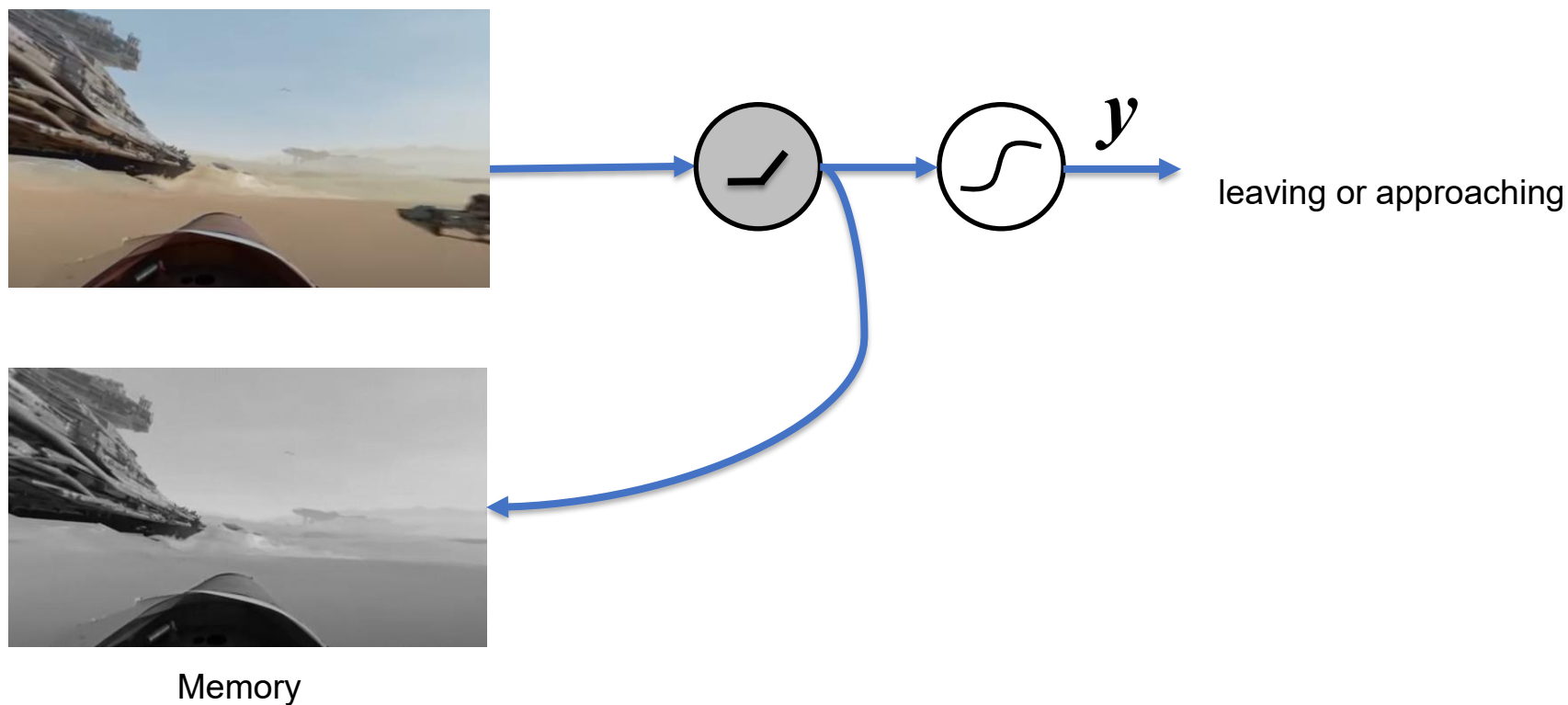




Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$



Memory?



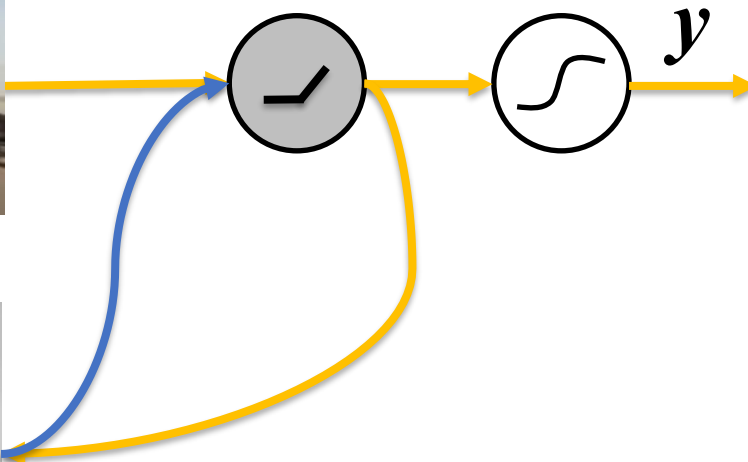


Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)
 - $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$



Memory as another input



leaving or approaching

Memory?





Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

	t0	t1	t2
Input:	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Output:

Memory?



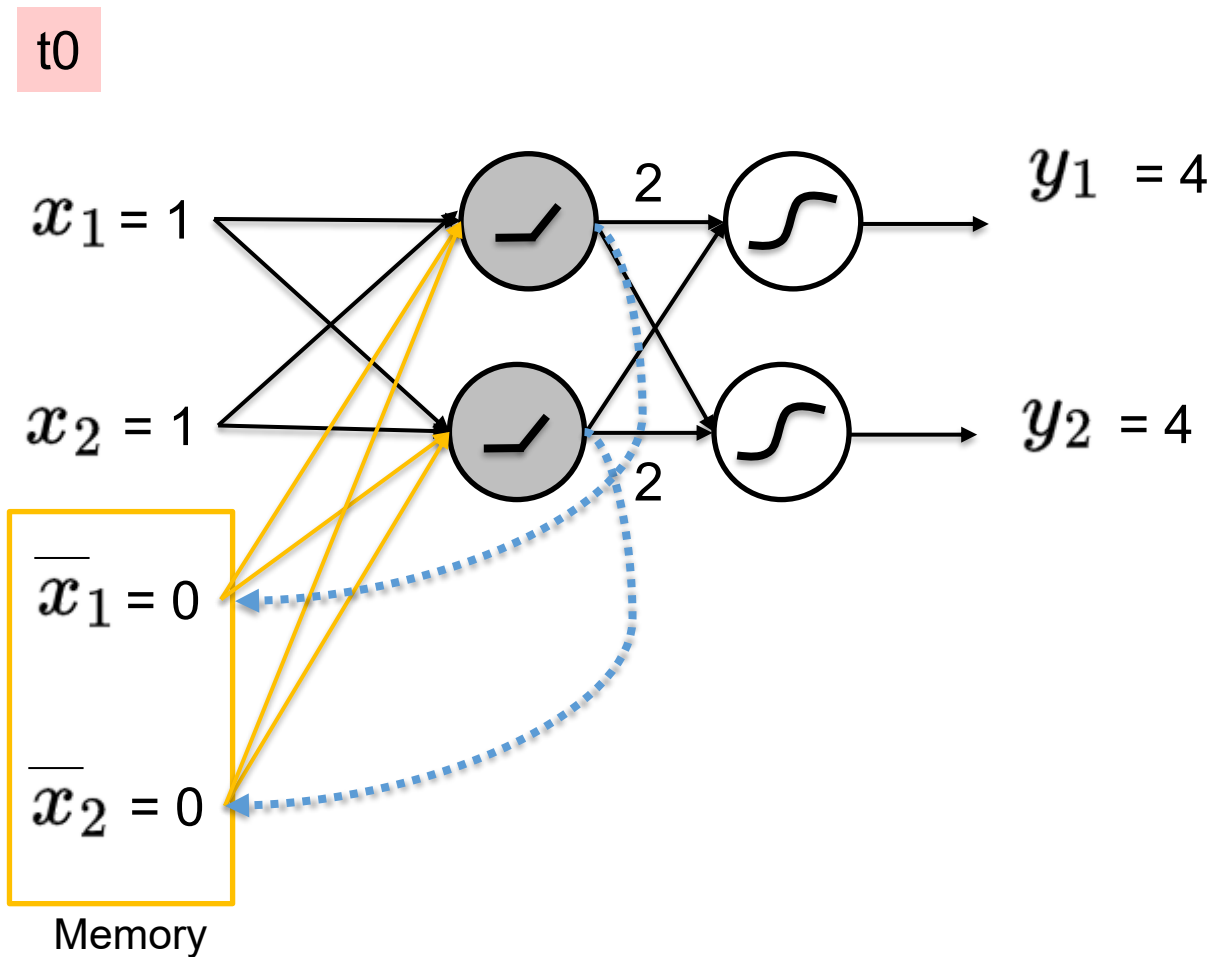


Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

	t0	t1	t2
Input:	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$
Output:	$\begin{bmatrix} 4 \\ 4 \end{bmatrix}$		



Memory?



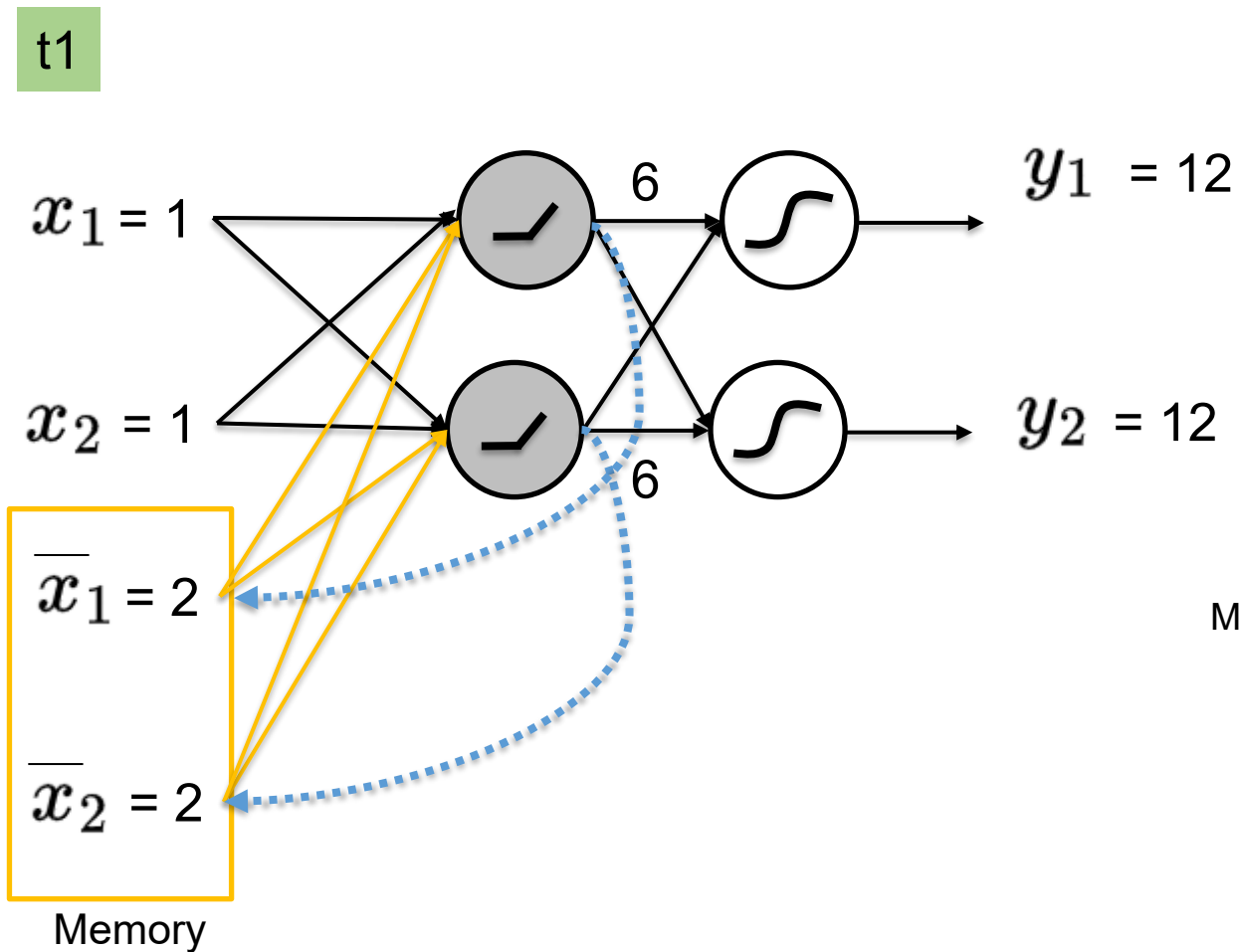


Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

	t0	t1	t2
Input:	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$
Output:	$\begin{bmatrix} 4 \\ 4 \end{bmatrix}$	$\begin{bmatrix} 12 \\ 12 \end{bmatrix}$	



Memory?



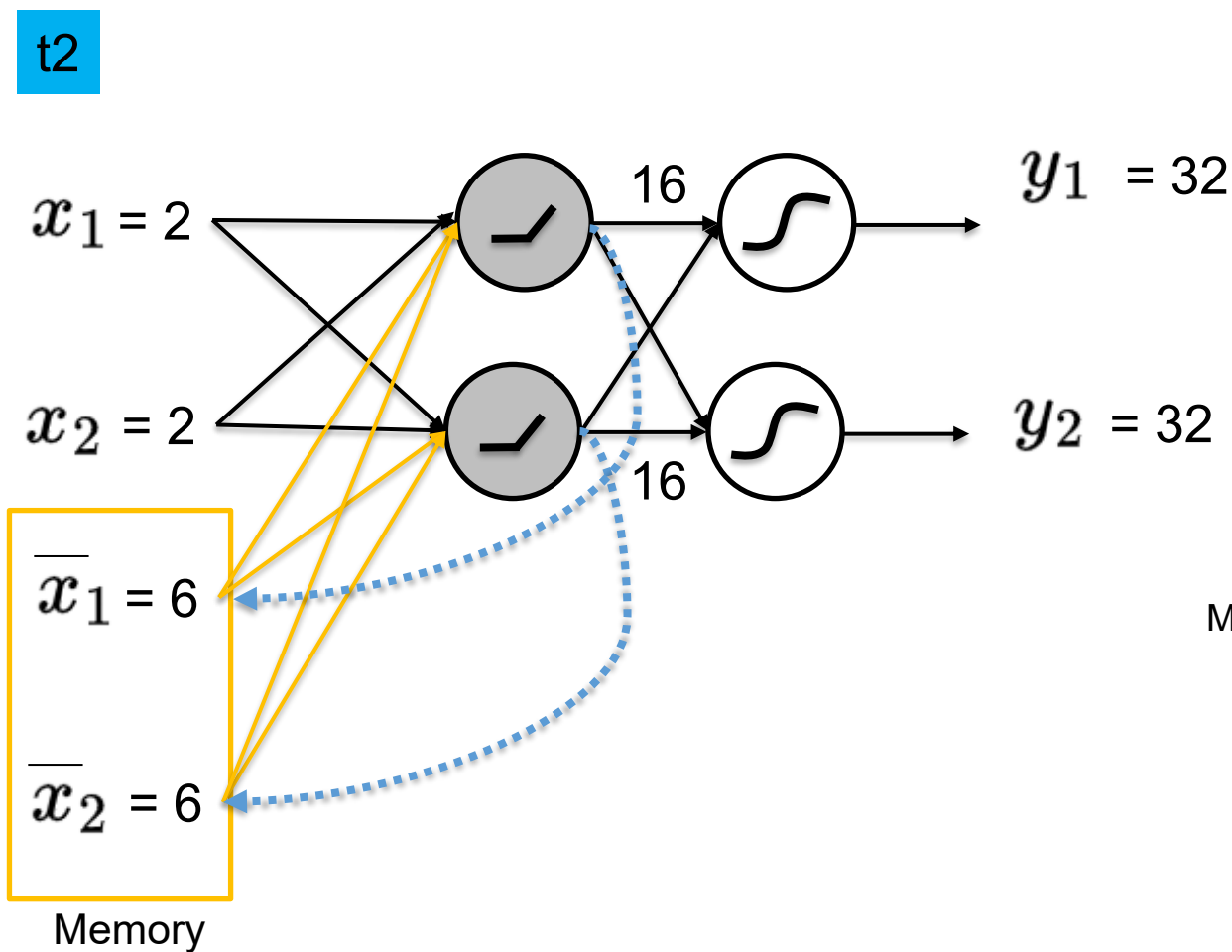


Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

	t0	t1	t2
Input:	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$
Output:	$\begin{bmatrix} 4 \\ 4 \end{bmatrix}$	$\begin{bmatrix} 12 \\ 12 \end{bmatrix}$	$\begin{bmatrix} 32 \\ 32 \end{bmatrix}$



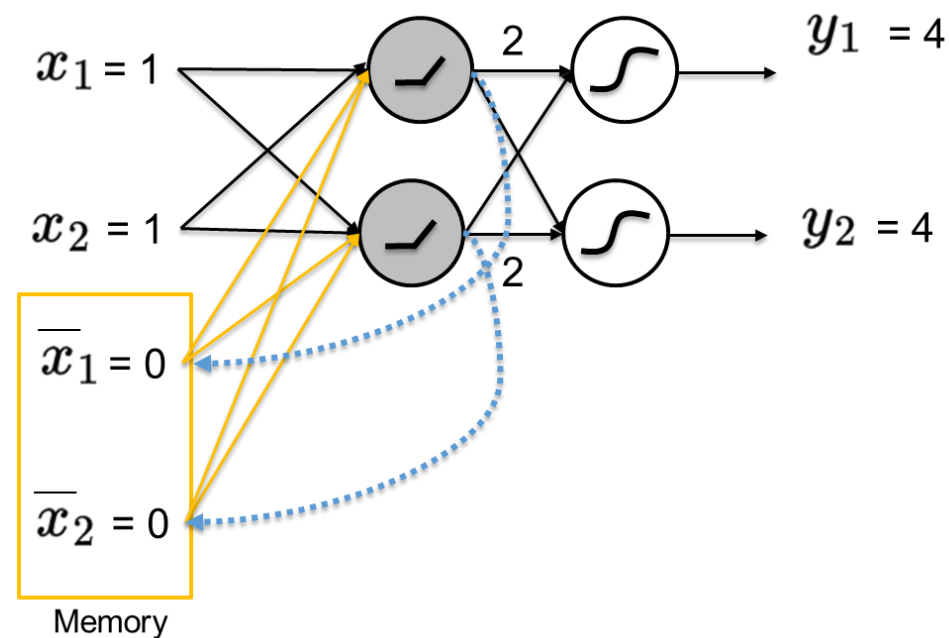
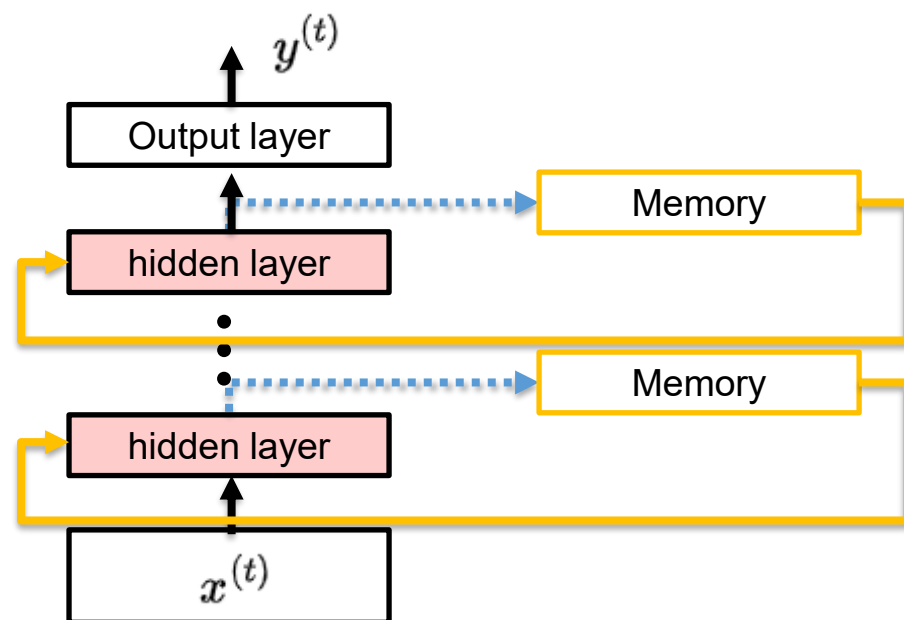
Memory?





Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)
 - $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

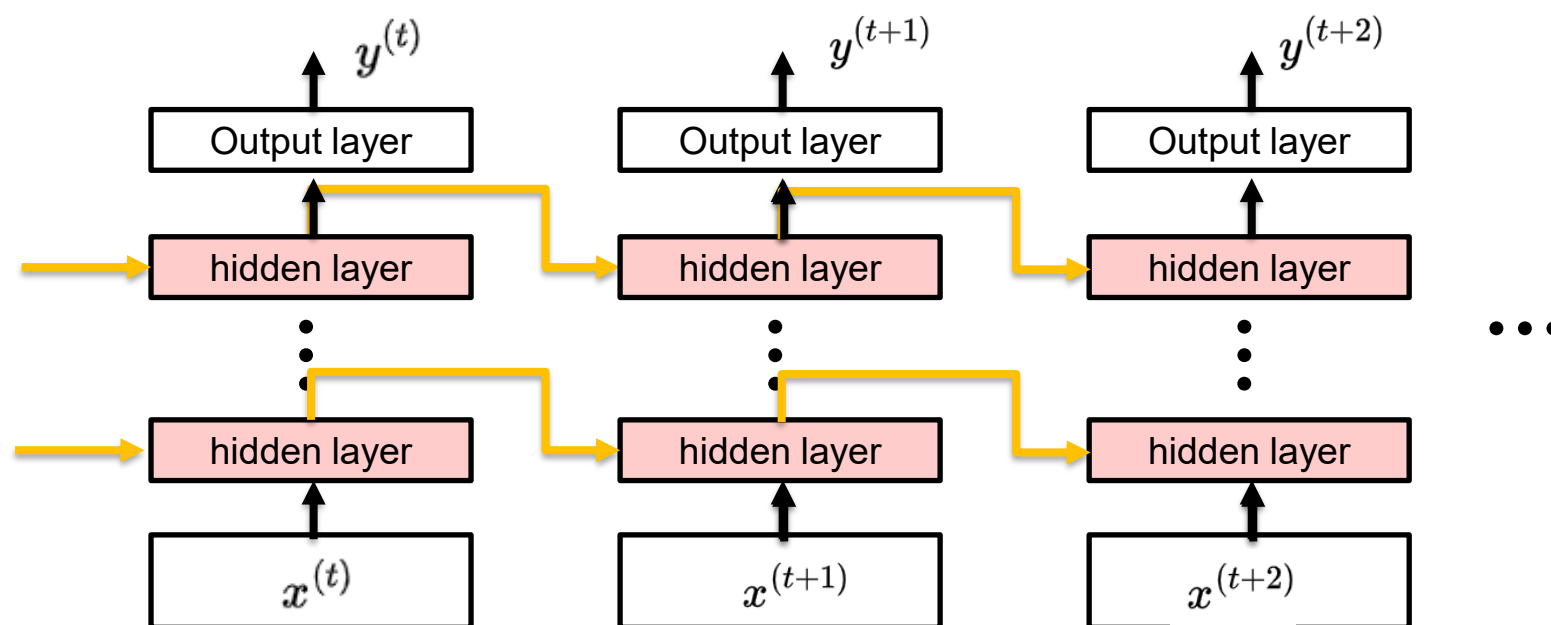




Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$



Sequence Model



國立清華大學
NATIONAL TSING HUA UNIVERSITY

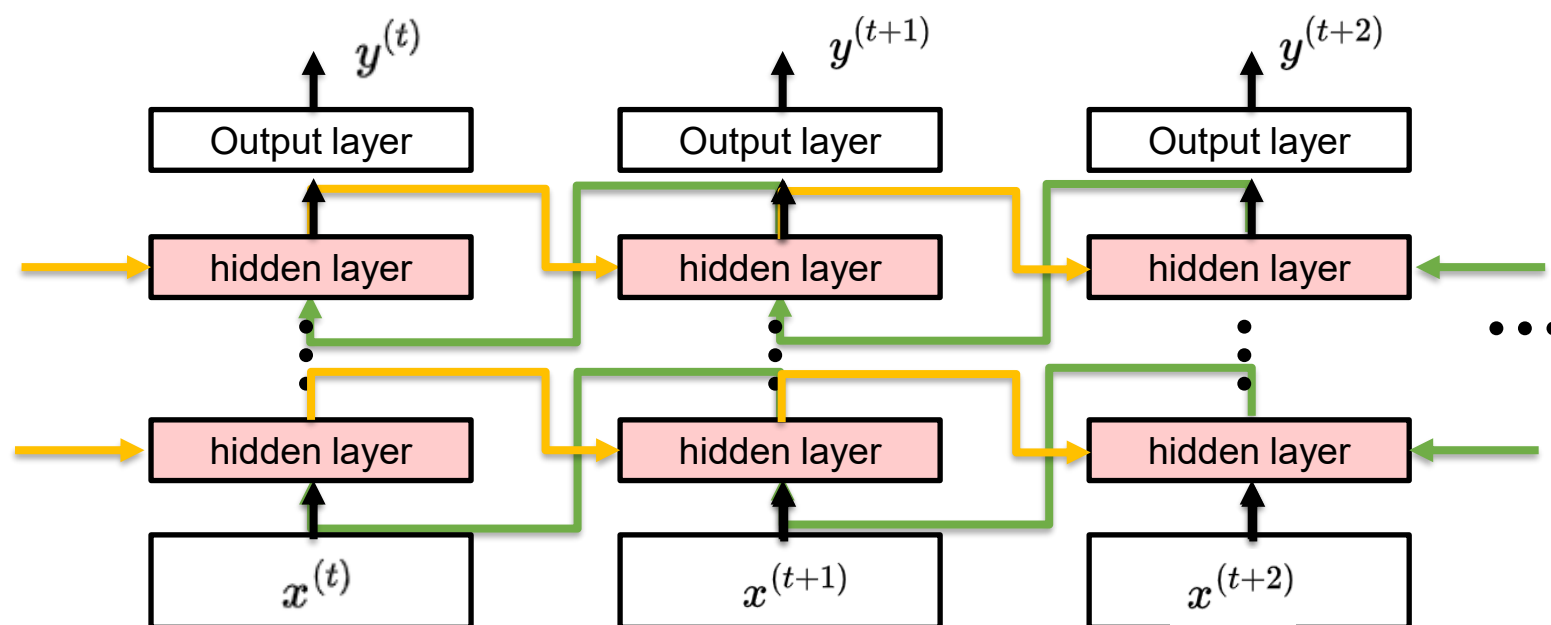




Sequence Model

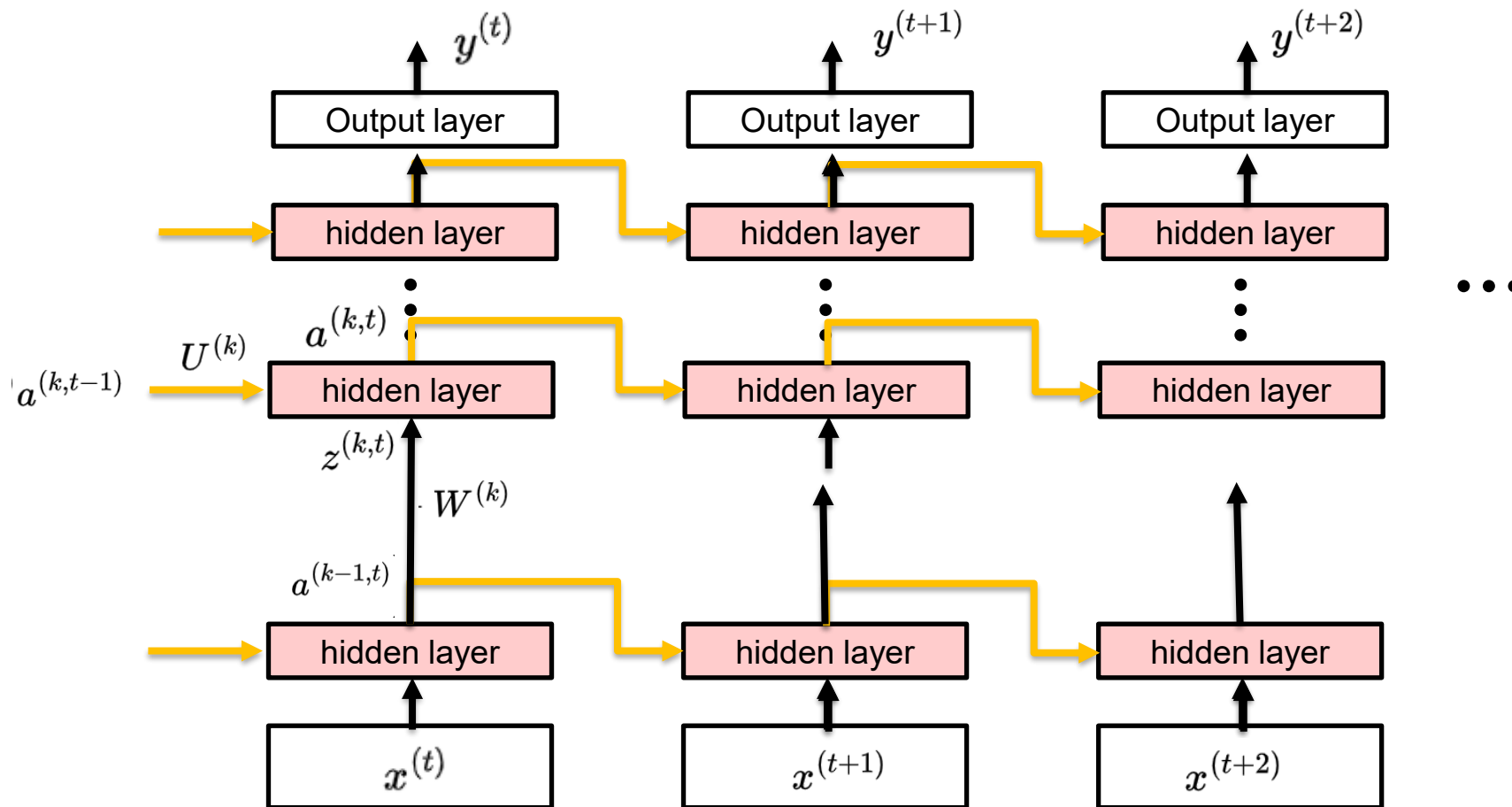
- Recurrent neural networks (Bidirectional RNN)

1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$



Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)
 1. $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$

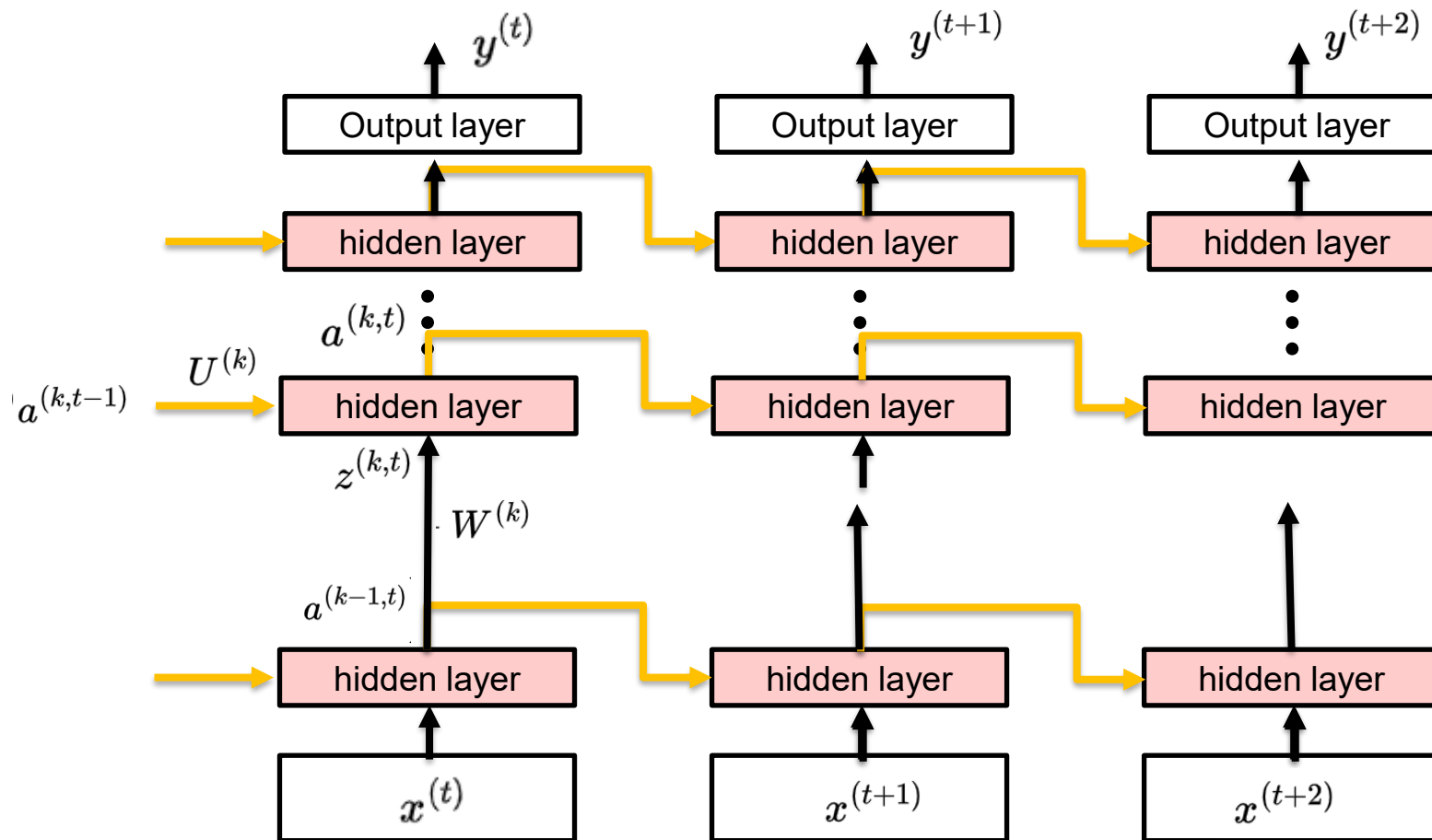


$$\begin{aligned} a^{(k,t)} &= activation \left(z^{(k,t)} \right) \\ &= activation \left(U^{(k)} a^{(k,t-1)} + W^{(k)} a^{(k-1,t)} \right) \end{aligned}$$



Sequence Model

- Recurrent neural networks (RNN, vanilla RNN)
 - $y^{(t)}$ depends on $x^{(1)}, \dots, x^{(t)}$



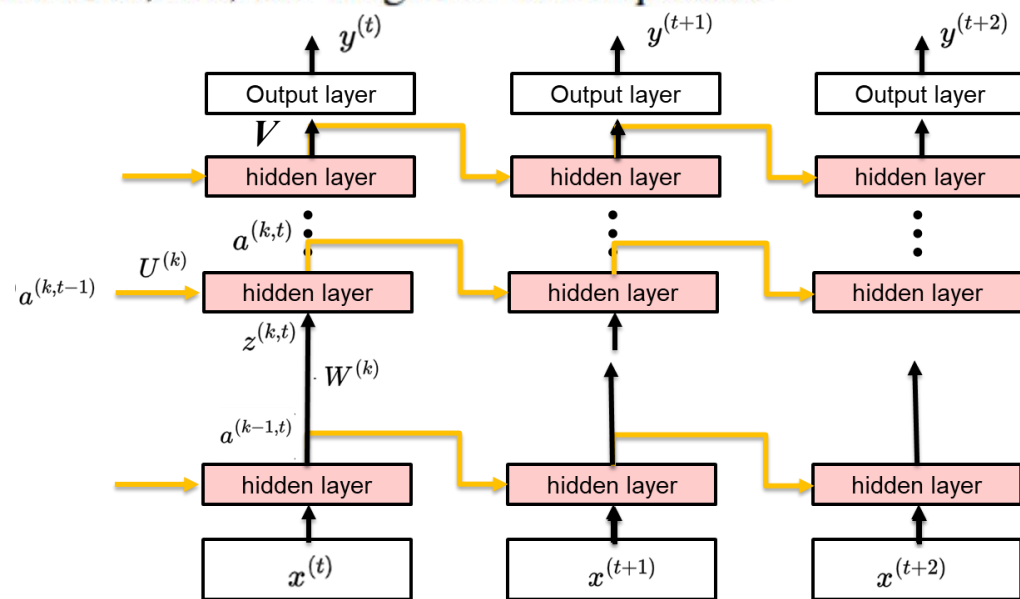
$$\begin{aligned} a^{(k,t)} &= \text{activation} \left(z^{(k,t)} \right) \\ &= \text{activation} \left(U^{(k)} a^{(k,t-1)} + W^{(k)} a^{(k-1,t)} \right) \end{aligned}$$

Weights, U and W , are shared across time instances.
Earlier representation is less important to current output.
...

Recurrent Neural Networks (10.2)

Recurrent Neural Networks (RNNs) operate on a sequence of input vectors $\mathbf{x}(t)$, where t denotes the time step, ranging from 1 to T . The output $y^{(t)}$ depends on the sequence $x^{(1)}, \dots, x^{(t)}$. In practice, recurrent networks often process minibatches of such sequences, with each sequence in the minibatch potentially having a different length t .

Assume the data points in the dataset $X^{(n)} = \{(x^{(n,t)}, y^{(n,t)})\}_n \in \mathbb{R}^{N \times (D,K) \times T}$ are independently and identically distributed (i.i.d.). For each sample n and time step t , there exists an input vector of size D and a corresponding output vector of size K . Here, (D, K) indicates a paired structure of input–output dimensions, and T denotes the *time horizon*, i.e., the length of the sequence.



In the RNN structure, the connections are parameterized as follows:

- **Input-to-hidden** connections are represented by the weight matrix U
- **Hidden-to-hidden** connections are represented by the weight matrix W
- **Hidden-to-output** connections are represented by the weight matrix V

U controls the temporal dependency by linking the previous hidden state to the current one.

W governs the feedforward transformation from the input (or lower layer) to the hidden activations.

V maps the final hidden state to the model output.

Unlike $U^{(k)}$ and $W^{(k)}$, the matrix V does not participate in temporal propagation. It is applied only once, at the final stage of computation, to project the hidden representation onto the output space. For this reason, V is sometimes treated separately or implicitly incorporated into the likelihood term during probabilistic formulation.



Recurrent Neural Networks (10.2)

At each time step t , the recurrent neural network (RNN) processes an input vector $\mathbf{x}^{(t)} \in \mathbb{R}^D$ and produces an output prediction $\mathbf{y}^{(t)} \in \mathbb{R}^K$. The hidden activation $\mathbf{a}^{(k,t)}$ at layer k and time step t is computed recursively according to the relation

$$\mathbf{a}^{(k,t)} = \text{act}\left(\mathbf{U}^{(k)}\mathbf{a}^{(k,t-1)} + \mathbf{W}^{(k)}\mathbf{a}^{(k-1,t)} + \mathbf{b}^{(k)}\right)$$

The nonlinear activation function is typically chosen as the hyperbolic tangent function (*tanh*) in RNNs.

- The *tanh* maps inputs to the range $(-1,1)$, allowing smoother gradient flow through time compared with the standard sigmoid activation.
- This helps mitigate vanishing gradient effects and enables the network to capture both positive and negative correlations in sequential data.

The model output at each time step t is computed by applying a linear mapping from the final hidden representation to the output space, expressed as

$$\mathbf{y}^{(t)} = g\left(\mathbf{V}\mathbf{a}^{(K,t)} + \mathbf{b}_y\right)$$

Additionally, the model's weights are shared across time instances, allowing the network to generalize its learning across different time steps within the sequence.

Recurrent Neural Networks (10.2)

To optimize the model parameters Θ , we apply the principle of maximum likelihood estimation (MLE), which is equivalent to minimizing the negative log-likelihood, or cost function, defined as

$$\begin{aligned} \arg \min_{\Theta} C(\Theta) &= \arg \min_{\Theta} -\log P(\mathbf{X} \mid \Theta) \\ &= \arg \min_{\Theta} -\sum_{n,t} \log P\left(\mathbf{y}^{(n,t)} \mid \mathbf{x}^{(n,t)}, \dots, \mathbf{x}^{(n,1)}, \Theta\right). \end{aligned}$$

Assuming that the output at each time step t follows a **Bernoulli distribution** parameterized by the model prediction, we can write

$$P\left(y^{(n,t)} = 1 \mid \mathbf{x}^{(n,t)}, \dots, \mathbf{x}^{(n,1)}\right) \sim \text{Bernoulli}\left(\rho^{(t)}\right)$$

where $\rho^{(t)} = P(y^{(n,t)} = 1 \mid \mathbf{x}^{(n,t)}, \dots, \mathbf{x}^{(n,1)})$ represents the predicted probability at time step t .

$$y^{(L,t)} = \rho^{(t)} \approx P(y^{(n,t)} = 1 \mid x^{(n,1)}, \dots, x^{(n,t)}; \Theta)$$

The predicted probability $\rho(t)$ is determined by the final hidden activation $\mathbf{a}^{(K,t)}$, which encodes the sequence history $\mathbf{x}^{(n,1)}, \dots, \mathbf{x}^{(n,t)}$.

The cost function can thus be expressed as

$$\begin{aligned} C^{(n,t)}(\Theta) &= \left(\mathbf{y}^{(K,t)}\right)^{y^{(n,t)}} \left(1 - \mathbf{y}^{(K,t)}\right)^{(1-y^{(n,t)})} \\ C^{(n,t)}(\Theta) &= \left(\mathbf{a}^{(K,t)}\right)^{y^{(n,t)}} \left(1 - \mathbf{a}^{(K,t)}\right)^{1-y^{(n,t)}} \end{aligned}$$



Recurrent Neural Networks (10.2)

Back-propagation through time (BPTT) by Stochastic Gradient Descent (SGD)

Parameter update:

$$\Theta^{(\tau+1)} \leftarrow \Theta^{(\tau)} - \eta \nabla_{\Theta} \sum_{n,t} c^{(n,t)},$$

where $c^{(n,t)} = C^{(n,t)}(\Theta^{(\tau)})$.

Gradient with respect to input-to-hidden weights:

$$\frac{\partial c^{(n,t)}}{\partial U_{i,j}^{(k)}} = \frac{\partial c^{(n,t)}}{\partial z_j^{(k,t)}} \cdot \frac{\partial z_j^{(k,t)}}{\partial U_{i,j}^{(k)}} = \delta_j^{(k,t)} \frac{\partial z_j^{(k,t)}}{\partial U_{i,j}^{(k)}}$$

Forward pass equations:

$$z^{(k,t)} \leftarrow \mathbf{U}^{(k)\top} \mathbf{a}^{(k,t-1)} + \mathbf{W}^{(k)\top} \mathbf{a}^{(k-1,t)},$$

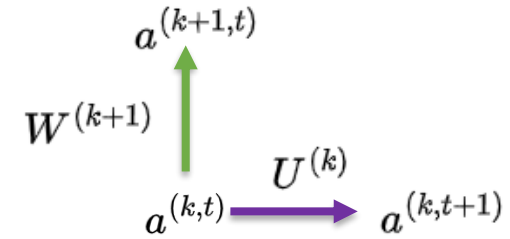
$$\mathbf{a}^{(k,t)} \leftarrow \text{act} \left(z^{(k,t)} \right),$$

$$\frac{\partial z_j^{(k,t)}}{\partial U_{i,j}^{(k)}} = a_i^{(k,t-1)}.$$

Earlier time shows at the shallower layer.

Backward pass equations:

$$\begin{aligned} \delta_j^{(k,t)} &= \frac{\partial c^{(n,t)}}{\partial z_j^{(k,t)}} = \frac{\partial c^{(n,t)}}{\partial a_j^{(k,t)}} \cdot \frac{\partial a_j^{(k,t)}}{\partial z_j^{(k,t)}}, \\ &= \frac{\partial c^{(n,t)}}{\partial a_j^{(k,t)}} \cdot \text{act}' \left(z_j^{(k,t)} \right), \\ &= \left(\sum_{\tau} \frac{\partial c^{(k,t)}}{\partial z_{\tau}^{(k+1,t)}} \cdot \frac{\partial z_{\tau}^{(k+1,t)}}{\partial a_j^{(k,t)}} + \sum_{\tau} \frac{\partial z_{\tau}^{(k,t)}}{\partial z_{\tau}^{(k+1,t)}} \cdot \frac{\partial z_{\tau}^{(k+1,t)}}{\partial a_j^{(k,t)}} \right) \text{act}' \left(z_j^{(k,t)} \right) \end{aligned}$$



The index τ labels all neurons that receive input from the current neuron $a_j^{(k,t)}$.

- all neurons in the **next layer** $a_{\tau}^{(k+1,t)}$
- all neurons in the **same layer at the next time** $a_{\tau}^{(k,t+1)}$.

$$\delta_j^{(k,t)} = \left(\sum_{\tau} \delta_{\tau}^{(k+1,t)} W_{j,\tau}^{(k+1)} + \sum_{\tau} \delta_{\tau}^{(k,t+1)} U_{j,\tau}^{(k)} \right) \text{act}' \left(z_j^{(k,t)} \right)$$

Sum over all τ in the next layer that depend on this neuron through W .

Sum over all τ in the next time step that depend on this neuron through U .



Backpropagation Through Time

The forward and backward pass equations provide the foundation for the computation of the network's activations and gradients across time steps.

Forward pass: For each n , forward pass is the same.

$$z^{(k,t)} \leftarrow \mathbf{U}^{(k)\top} \mathbf{a}^{(k,t-1)} + \mathbf{W}^{(k)\top} \mathbf{a}^{(k-1,t)}$$

$$\mathbf{a}^{(k,t)} \leftarrow \text{act} \left(z^{(k,t)} \right)$$

Backward pass: For each n , backward pass is computed by t .

$$\delta^{(k,t)} \leftarrow \text{act}' \left(z^{(k,t)} \right) \odot \left(\mathbf{U}^{(k)} \delta^{(k,t+1)} + \mathbf{W}^{(k+1)} \delta^{(k+1,t)} \right)$$

Challenges

The dependencies between $\mathbf{a}^{(k,i)}$ (resp. $\delta^{(k,i)}$) and $\mathbf{a}^{(k,j)}$ (resp. $\delta^{(k,j)}$) are maintained by $\left(\mathbf{U}^{(k)} \right)^{(i-j)}$.

Ignoring the activation function and depth, we have:

$$\mathbf{a}^{(k,j)} = \left(\mathbf{U}^{(k)\top} \right)^{j-i} \mathbf{a}^{(k)} \quad \text{and} \quad \delta^{(k,t)} = \left(\mathbf{U}^{(k)} \right)^{j-i} \delta^{(k,f)}$$

Ignoring the activation function and depth, we have:

$$\mathbf{a}^{(k,j)} = \left(\mathbf{U}^{(k)\top} \right)^{j-i} \mathbf{a}^{(k)} \quad \text{and} \quad \delta^{(k,t)} = \left(\mathbf{U}^{(k)} \right)^{j-i} \delta^{(k,f)}$$

If $\mathbf{a}^{(k,i)}$ and $\mathbf{a}^{(k,j)}$ are far apart in time, their long-term dependency may cause optimization issues. Given the eigendecomposition:

$$\mathbf{U}^{(k)} = \mathbf{Q} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \lambda_{D(i)} \end{bmatrix} \mathbf{Q}^\top$$

Then,

$$\left(\mathbf{U}^{(k)} \right)^{j-i} = \mathbf{Q} \begin{bmatrix} \lambda_1^{j-i} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \lambda_{D(k)}^{j-i} \end{bmatrix} \mathbf{Q}^\top$$

When $j-i$ is large, and λ is either very large or very small (which results in a very flat or steep cost surface C), this can cause exploding or vanishing gradients, making gradient-based optimization very difficult.

$$\lambda_s = 1.01 \Rightarrow \lambda_s^{1000} \approx 20,000$$

$$\lambda_s = 0.99 \Rightarrow \lambda_s^{1000} \approx 0$$

$$\lambda_s = 1.01 \Rightarrow \lambda_s^{1000} \approx 20,000$$

$$\lambda_s = 0.99 \Rightarrow \lambda_s^{1000} \approx 0$$

Backpropagation Through Time

- $\delta^{(k,t)} \leftarrow \text{act}'(z^{(k,t)}) \odot (U^{(k)} \delta^{(k,t+1)} + W^{(k+1)} \delta^{(k+1,t)})$

Then,

$$(U^{(k)})^{j-i} = Q \begin{bmatrix} \lambda_1^{j-i} & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_{D^{(k)}}^{j-i} \end{bmatrix} Q^\top$$

When $j-i$ is large, and λ is either very large or very small (which results in a very flat or steep cost surface C), this can cause exploding or vanishing gradients, making gradient-based optimization very difficult.

- Adam – Adaptive Moment Estimation**

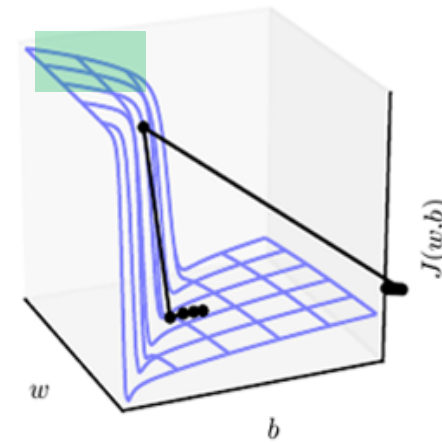
Then, The Adam optimizer combines the advantages of both **RMSProp** and Momentum by maintaining moving averages of the gradients and their second moments, which use exponential decay rates to apply bias corrections.

$$\mathbf{v}^{(\tau+1)} = \lambda_1 \mathbf{v}^{(\tau)} + (1 - \lambda_1) \nabla E_n^{(\tau)}$$

$$\mathbf{r}^{(\tau+1)} = \lambda_2 \mathbf{r}^{(\tau)} + (1 - \lambda_2) \nabla E_n^{(\tau)} \odot \nabla E_n^{(\tau)}$$

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \frac{\eta}{\sqrt{\mathbf{r}^{(\tau+1)}}} \mathbf{v}^{(\tau+1)}$$

Without clipping



With clipping

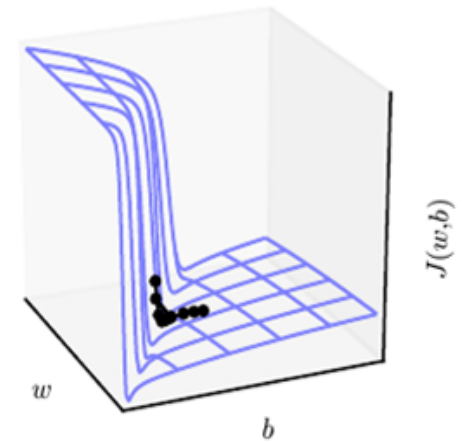


Fig. 17: In regions where the cost surface has extremely steep cliffs (i.e., very large or very small gradients), gradient clipping helps by limiting the gradient values to a predefined threshold. This makes gradient descent more stable and allows it to converge more effectively, avoiding the issues of exploding gradients.